Car Detection System: Faster R-CNN vs YOLO

📖 TABLE OF CONTENTS

1. Project Overview
2. What is Object Detection?
3. Algorithms Explained
4. Project Architecture
5. Dataset Description
6. Implementation Steps
7. Results & Analysis
8. Conclusion
9. Future Work
10. Appendix

1. PROJECT OVERVIEW

1.1 Problem Statement

In today's world, autonomous vehicles, traffic monitoring systems, and surveillance cameras need to automatically detect cars in images and videos. Doing this manually is impossible for large-scale systems. We need computers that can look at images and tell us: **"Where are the cars?"**

This project solves this problem by building a **car detection system** using two state-of-the-art deep learning algorithms.

1.2 Objectives

The main goals of this project are:

Objective	Description
Detect Cars	Identify and locate cars in images with bounding boxes
Compare Algorithms	Compare Faster R-CNN vs YOLO on accuracy and speed
User-Friendly GUI	Build an easy-to-use interface for non-technical users
Provide Metrics	Show accuracy, precision, recall, and F1-score
Make Deployable	Create a standalone EXE that anyone can run

1.3 Tools & Technologies Used

Tool	Purpose
Python 3.8	Programming language
PyTorch 2.0	Deep learning framework
MMDetection	Object detection toolbox
Tkinter	GUI development
OpenCV	Image processing
CUDA 11.8	GPU acceleration (NVIDIA)
PyInstaller	EXE creation

1.4 Team/Author Information

- **Project Type:** Mini Project / Academic Research
- **Domain:** Computer Vision / Deep Learning
- **Year:** 2026

2. WHAT IS OBJECT DETECTION?

2.1 Definition

Object detection is a computer vision technique that **finds and identifies objects** in images or videos. It answers two questions:

1. **WHAT** is in the image? (Classification)
2. **WHERE** is it located? (Localization)

2.2 Difference: Classification vs Detection vs Segmentation

```
┌──────────────────────────────────────────────────────────────────┐
│ CLASSIFICATION        DETECTION           SEGMENTATION           │
│ "What is this?"       "What & Where?"     "Every pixel"          │
│                                                                  │
│ [CAT]                 [CAT]               [CAT]                  │
│ 100%                 ┌─────┐              ▀▀▀▀▀▀                 │
│                      │ CAT │              ▀▀   ▀▀                 │
│                      └─────┘              ▀▀▀▀▀▀                 │
│                                                                  │
│ One label per image   Box around object   Pixel-level mask       │
```

2.3 Real-world Applications

Application	Use Case
Self-driving Cars	Detect pedestrians, vehicles, traffic signs
Surveillance	Detect intruders, suspicious activity
Medical Imaging	Detect tumors, fractures
Retail	Count customers, track inventory
Agriculture	Detect diseases in crops
Security	Face detection, weapon detection

2.4 Challenges in Object Detection

Challenge	Description
Different Sizes	Cars can be small or large in the image
Different Angles	Side view, front view, top view
Partial Occlusion	Cars hidden behind trees or other cars
Lighting Conditions	Day, night, shadows, rain
Background Clutter	Buildings, roads, trees
Real-time Requirements	Need fast processing for video

3. ALGORITHMS EXPLAINED (DEEP DIVE)

3.1 FASTER R-CNN

What is R-CNN? (History)

The R-CNN family evolved over time:

Version	Year	Key Idea	Problem
R-CNN	2014	Use selective search for regions	Very slow (50 sec/image)

Fast R-CNN	2015	Shared computation	Still slow due to selective search
Faster R-CNN	2016	**Learnable region proposals**	Fast and accurate!

How Faster R-CNN Works

Faster R-CNN works in **two stages**:

```
INPUT IMAGE
      |
      ▼
┌──────────────────────┐
│ BACKBONE (ResNet50) │
│ Extract Features    │
└──────────────────────┘
      |
      ▼
┌──────────────────────────────┐
│ STAGE 1: REGION PROPOSAL    │
│ NETWORK (RPN)               │
│ "Where might cars be?"      │
└──────────────────────────────┘
      |
      ▼
~2000 Region Proposals
      |
      ▼
┌──────────────────────────────┐
│ STAGE 2: CLASSIFICATION     │
│ "What is in each region?"   │
└──────────────────────────────┘
      |
      ▼
```

Final Detections with Boxes

Step 1: Backbone (ResNet50)

The backbone is a pre-trained neural network that **extracts features** from the image.

- **What it does:** Converts image into feature maps
- **Why ResNet50:** Good balance of speed and accuracy

- **How it works:** 50 layers deep with skip connections

Step 2: Region Proposal Network (RPN)

The RPN slides over the feature map and asks: **"Could there be a car here?"**

- For each location, it places **9 anchor boxes** (different sizes and shapes)
- Each anchor box gets:
  - **Objectness score:** Is there any object?
  - **Box coordinates:** Where is it?
- It keeps the **top 2000** proposals

Step 3: ROI Pooling

All proposals have different sizes. ROI Pooling **makes them all the same size** so they can be processed together.

Step 4: Classification & Bounding Box Regression

Finally, each proposal is classified:

- **Class:** "Car" or "Background"
- **Box refinement:** Adjust coordinates to be more accurate

Advantages & Disadvantages

Advantages	Disadvantages
✅ Very high accuracy	❌ Slower than YOLO
✅ Good for small objects	❌ Two-stage design is complex
✅ Well-tested and reliable	❌ Requires more memory
✅ Good for static images	❌ Not ideal for real-time video

When to use Faster R-CNN

- ✅ When **accuracy is the top priority**
- ✅ For **forensic analysis** (no mistakes allowed)
- ✅ For **medical imaging** (high precision needed)
- ✅ When speed is **not critical** (2-3 FPS is fine)

3.2 YOLO (You Only Look Once)

Philosophy: Single Shot Detection

YOLO's philosophy is simple: **Look at the image once and detect everything in a single pass.**

No region proposals. No second stage. Just **one forward pass** through the network.

How YOLO Works

```
INPUT IMAGE (608×608)
        |
        ▼
┌──────────────────────┐
| Divide into Grid     |
|   19×19 = 361 cells  |
└──────────────────────┘
        |
        ▼
┌──────────────────────┐
| Single CNN Pass      |
| Darknet-53 Backbone  |
└──────────────────────┘
        |
        ▼
┌──────────────────────┐
```

```
│ For Each Cell:      │
│ Predict 3 boxes     │
│ • x, y (center)     │
│ • w, h (size)       │
│ • confidence score  │
│ • class probability │
└─────────────────────┘
        │
        ▼
~1083 predictions (361×3)
        │
        ▼
┌─────────────────────┐
│ Non-Maximum         │
│ Suppression (NMS)   │
│ Remove duplicates   │
└─────────────────────┘
        │
        ▼
```

Final Detections

Step 1: Grid Division

The image is divided into an **S×S grid** (19×19 = 361 cells).

- Each cell is responsible for detecting objects whose center falls in that cell

Step 2: Bounding Box Prediction

Each cell predicts **B bounding boxes** (typically 3 or 5).

For each box, YOLO predicts:

- **x, y:** Center coordinates relative to the cell
- **w, h:** Width and height relative to the image
- **confidence:** How sure the model is that an object exists

Step 3: Class Probability

For each cell, YOLO predicts class probabilities: **"If there is an object, what is it?"**

In our project, only one class: **"car"**

Step 4: Non-Maximum Suppression (NMS)

After prediction, we have many overlapping boxes. NMS removes duplicates:

1. Sort boxes by confidence score
2. Take the highest confidence box
3. Remove boxes with high overlap (IoU > 0.5)
4. Repeat until no boxes left

YOLO Versions

Version	Year	Key Improvement
YOLOv1	2016	First version, 45 FPS
YOLOv2	2017	Better accuracy, 9000 classes
YOLOv3	**2018**	**Used in this project**
YOLOv4	2020	Mosaic augmentation, CSPDarknet
YOLOv5	2020	PyTorch implementation
YOLOv8	2023	Anchor-free detection

Advantages & Disadvantages

Advantages	Disadvantages
✅ Very fast (20-30 FPS)	❌ Less accurate than Faster R-CNN
✅ Real-time capable	❌ Struggles with small objects
✅ Simple architecture	❌ More false positives
✅ Good for video	❌ Less precise localization

When to use YOLO

- ✅ When **speed is the top priority** (real-time applications)
- ✅ For **video surveillance** (needs 30 FPS)
- ✅ For **self-driving cars** (must react quickly)
- ✅ When **computational resources are limited**

3.3 COMPARISON: Faster R-CNN vs YOLO

Speed vs Accuracy Trade-off

```
HIGH ACCURACY ◀──────────────────────────▶ HIGH SPEED
    |                        |
    ▼                        ▼
┌──────────────────┐      ┌──────────────────┐
| FASTER R-CNN     |      |     YOLO         |
|                  |      |                  |
| Accuracy: 95%    |      | Accuracy: 52%    |
| Speed: 2-3 FPS   |      | Speed: 30 FPS    |
|                  |      |                  |
```

```
| "I'm very sure |              | "I'm fast but  |
| but I'm slow" |              | sometimes     |
|              |              | wrong"       |

└──────────────┘              └──────────────┘
```

Comparison Table

Aspect	Faster R-CNN	YOLO
Approach	Two-stage (Propose + Classify)	One-stage (Direct regression)
Accuracy (mAP@0.5)	**94.9%**	52.0%
Speed	2-3 FPS	20-30 FPS
Best for	Static images, high accuracy	Video, real-time
Memory usage	Higher (~4GB)	Lower (~2GB)
Small objects	Better	Worse
Training time	1.5 hours	2 hours

Use Cases Summary

Scenario	Recommended Model
Police evidence analysis	Faster R-CNN (no mistakes allowed)
Real-time traffic monitoring	YOLO (needs speed)

Autonomous driving	YOLO (must react quickly)
Medical imaging	Faster R-CNN (accuracy critical)
Retail customer counting	YOLO (real-time processing)
Satellite image analysis	Faster R-CNN (can process offline)

4. PROJECT ARCHITECTURE

4.1 Complete Folder Structure

```
Car_Detection/                  ← Main project folder
|
├── 📁 app_gui/                 ← GUI application
|   ├── car_detection_app.py    ← Main GUI (Tkinter)
|   └── detection_engine.py     ← Model loading & detection
|
├── 📁 configs/                 ← Model configurations
|   ├── 📁 faster_rcnn/
|   |   └── faster_rcnn_car.py  ← Faster R-CNN settings
|   └── 📁 yolo/
|       └── yolo_car.py         ← YOLO settings
|
├── 📁 tools/                   ← Helper scripts
|   ├── fix_yolo_validation.py  ← Data preparation
|   ├── rcnn_test.py            ← Test Faster R-CNN
|   ├── yolo_detect.py          ← Test YOLO
|   ├── compare_models_final.py ← Compare both models
|   ├── To_find_frcnn_accuracy.py ← Calculate FRCNN accuracy
```

```
│   └── To_find_yolo_accuracy.py      ← Calculate YOLO accuracy
│
├── 📁 data/                          ← Dataset (user downloads)
│   └── 📁 car_dataset/
│       ├── 📁 training_images/       ← 284 training images
│       ├── 📁 testing_images/        ← 71 testing images
│       └── train_solution_bounding_boxes.csv ← Annotations
│
├── 📁 work_dirs/                     ← Trained models
│   ├── 📁 faster_rcnn_car/
│   │   └── epoch_12.pth              ← Trained Faster R-CNN
│   └── 📁 yolo_car/
│       └── epoch_50.pth              ← Trained YOLO
│
├── 📁 mmdetection/                   ← MMDetection library (cloned)
│
├── 📁 results/                       ← Detection output images
│
├── 📄 requirements.txt               ← All Python dependencies
├── 📄 .gitignore                     ← Files to exclude from GitHub
```

└── 📄 installation_guide.html ← User installation guide

4.2 Module Description

app_gui/ - GUI Application

File	Purpose
`car_detection_app.py`	Main window with controls, image display, metrics table

`detection_engine.py`	Loads models, runs inference, draws bounding boxes

configs/ - Model Configurations

File	Purpose
`faster_rcnn/faster_rcnn_car.py`	Batch size (2), epochs (12), learning rate (0.02)
`yolo/yolo_car.py`	Batch size (4), epochs (50), Darknet-53 backbone

tools/ - Utility Scripts

Script	Purpose
`fix_yolo_validation.py`	Converts CSV annotations to COCO JSON format
`rcnn_test.py`	Test Faster R-CNN on single image
`yolo_detect.py`	Test YOLO on single image
`compare_models_final.py`	Side-by-side comparison
`To_find_frcnn_accuracy.py`	Calculate precision, recall, F1 for FRCNN
`To_find_yolo_accuracy.py`	Calculate precision, recall, F1 for YOLO

4.3 Data Flow Diagram

```
┌──────────────────────────────────────┐
│          USER INTERFACE              │
│  (Select Algorithm, Upload Image, Run)  │
└──────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────┐
│          DETECTION ENGINE            │
│  (Loads Models, Runs Inference)      │
└──────────────────────────────────────┘
                  │
┌─────────────────┼─────────────────┐
│                 │                 │
▼                 ▼                 ▼
┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│ Faster R-CNN │  │   YOLO       │  │ Comparison   │
│   Model      │  │   Model      │  │ (Both)       │
└──────────────┘  └──────────────┘  └──────────────┘
      │                 │                 │
      └─────────────────┼─────────────────┘
                        ▼
┌──────────────────────────────────────┐
│          OUTPUT                      │
│  • Image with Bounding Boxes         │
│  • Car Count                         │
│  • Inference Time                    │
│  • mAP, Precision, Recall, F1-Score  │
```

4.4 Training Pipeline

```
┌──────────────────────────────────────────────────────────┐
│                    TRAINING PIPELINE                     │
└──────────────────────────────────────────────────────────┘

  Dataset (284 images)        Config File
       │                           │
       ▼                           ▼
  ┌──────────────────────────────────────────────┐
  │            DataLoader                        │
  │  • Load images                               │
  │  • Apply augmentations (resize, flip, rotate)  │
  │  • Batch size = 2 (FRCNN) or 4 (YOLO)        │
  └──────────────────────────────────────────────┘
                  │
                  ▼
  ┌──────────────────────────────────────────────┐
  │          Forward Pass                        │
  │  • Image → Backbone → Feature Maps           │
  │  • RPN → Proposals (FRCNN)                   │
  │  • Detections (YOLO)                         │
  └──────────────────────────────────────────────┘
                  │
                  ▼
  ┌──────────────────────────────────────────────┐
  │          Calculate Loss                      │
  │  • Classification loss                       │
  │  • Bounding box regression loss              │
```

```
└──────────────────────────────────────────┘
                  |
                  ▼
┌──────────────────────────────────────────┐
|          Backward Pass                   |
| • Compute gradients                      |
| • Update weights (SGD optimizer)         |
└──────────────────────────────────────────┘
                  |
                  ▼
┌──────────────────────────────────────────┐
|          Validation (Every Epoch)        |
| • Run on validation set (71 images)      |
| • Calculate mAP, Precision, Recall       |
└──────────────────────────────────────────┘
                  |
                  ▼
┌──────────────────────────────────────────┐
|          Save Checkpoint                 |
| • Save model after each epoch            |
| • Final model: epoch_12.pth / epoch_50.pth |
```

5. DATASET DESCRIPTION

5.1 Source

Dataset Name: Car Object Detection Dataset

Source: Kaggle

Link: https://www.kaggle.com/datasets/sshikamaru/car-object-detection

5.2 Dataset Statistics

Category	Count
Total Images	355
Training Images	284 (80%)
Validation Images	71 (20%)
Total Car Annotations	559
Average Cars per Image	1.57
Image Format	JPG
Annotation Format	COCO (converted from CSV)

5.3 Sample Images Distribution

Training Images: 284 images, 440 cars

Validation Images: 71 images, 119 cars

Image IDs: vid_4_xxxx.jpg and vid_5_xxxx.jpg

- vid_4_* : Mostly side views of cars

- vid_5_* : Different angles and scenarios

5.4 Annotation Format (COCO)

COCO (Common Objects in Context) is a standard format for object detection:

```
{
  "images": [
    {
      "id": 1,
      "file_name": "vid_4_13680.jpg",
      "width": 1920,
      "height": 1080
    }
  ],
  "annotations": [
    {
      "id": 1,
      "image_id": 1,
      "category_id": 1,
      "bbox": [100, 150, 200, 250],
      "area": 50000,
      "iscrowd": 0
    }
  ],
  "categories": [
    {
      "id": 1,
      "name": "car"
    }
  ]
}
```

5.5 Data Preprocessing Steps

Step 1: Download raw dataset from Kaggle

↓

Step 2: Extract to data/car_dataset/

↓

Step 3: Run fix_yolo_validation.py

↓

Step 4: Convert CSV to COCO JSON

↓

Step 5: Create train/val split (80/20)

↓

Step 6: Generate instances_train_fixed.json

↓

Step 7: Generate instances_val_fixed.json

↓

Step 8: Ready for training!

5.6 Data Augmentation

During training, we apply these augmentations:

Augmentation	Purpose
Random Horizontal Flip	50% chance to flip image (handles left/right views)
Resize	Scale images to 608×608 (YOLO) or 1333×800 (FRCNN)
Color Jitter	Small brightness/contrast changes (handles lighting)

6. IMPLEMENTATION STEPS

6.1 Step-by-Step Setup Guide

Step 1: Install Anaconda

Download from anaconda.com and install with "Add to PATH" option

Step 2: Create Virtual Environment

conda create --name mmdet_car python=3.8 -y

conda activate mmdet_car

Step 3: Install PyTorch (with GPU support)

pip install torch==2.0.0 torchvision==0.15.0 torchaudio==2.0.0 --index-url https://download.pytorch.org/whl/cu118

Step 4: Install MMDetection

pip install -U openmim

mim install mmengine

mim install mmcv==2.0.0

git clone https://github.com/open-mmlab/mmdetection.git

cd mmdetection

pip install -v -e .

cd ..

Step 5: Install Dependencies

pip install opencv-python pandas scikit-learn scipy matplotlib tabulate tqdm pycocotools

Step 6: Prepare Dataset

```
python tools/fix_yolo_validation.py
```

Step 7: Train Faster R-CNN

```
python mmdetection/tools/train.py configs/faster_rcnn/faster_rcnn_car.py --work-dir
work_dirs/faster_rcnn_car
```

Step 8: Train YOLO

```
python mmdetection/tools/train.py configs/yolo/yolo_car.py --work-dir work_dirs/yolo_car
```

Step 9: Run GUI

```
python app_gui/car_detection_app.py
```

6.2 Training Process

Faster R-CNN Training (12 Epochs)

Epoch	Training Loss	Validation mAP@0.5	Time
1	0.62	73.6%	7 min
2	0.25	85.0%	7 min
3	0.18	91.6%	7 min
4	0.15	89.9%	7 min
5	0.12	94.1%	7 min
6	0.10	94.6%	7 min
7	0.09	95.4%	7 min

8	0.08	93.7%	7 min
9	0.07	94.9%	7 min
10	0.06	94.9%	7 min
11	0.06	94.1%	7 min
12	0.05	**94.9%**	7 min

Total Training Time: ~1.5 hours

YOLO Training (50 Epochs)

Epoch Range	Validation mAP@0.5
1-10	19.3% → 35.0%
11-20	35.0% → 45.0%
21-30	45.0% → 50.0%
31-40	50.0% → 52.0%
41-50	52.0% → 52.0%

Total Training Time: ~2 hours

6.3 Hyperparameters

Faster R-CNN Hyperparameters

Parameter	Value
Backbone	ResNet50 (pretrained on ImageNet)
Batch Size	2
Learning Rate	0.02
Optimizer	SGD with momentum (0.9)
Weight Decay	0.0001
Epochs	12
Input Size	1333×800
Anchor Scales	[8]

YOLO Hyperparameters

Parameter	Value
Backbone	Darknet-53
Batch Size	4

Learning Rate	0.001
Optimizer	SGD with momentum (0.9)
Weight Decay	0.0005
Epochs	50
Input Size	608×608
Grid Size	19×19

6.4 Monitoring Training

During training, you see output like:

```
Epoch(train) [1][50/142] loss: 0.62 acc: 97.46% time: 2.96s
Epoch(val) [1][71/71] bbox_mAP: 0.278 bbox_mAP_50: 0.736
```

What this means:

- `loss`: How wrong the model is (lower = better)
- `acc`: Classification accuracy (higher = better)
- `bbox_mAP`: Average precision (higher = better)
- `bbox_mAP_50`: Precision at 50% overlap (higher = better)

---

7. RESULTS & ANALYSIS

7.1 Model Performance Metrics

What Each Metric Means

Metric	What It Measures	Formula
mAP	Average precision across all IoU thresholds	Average of AP at IoU=0.5 to 0.95
mAP@0.5	Precision when box overlap ≥50%	Standard object detection metric
Precision	Of all detections, how many were correct?	TP / (TP + FP)
Recall	Of all cars, how many did we find?	TP / (TP + FN)
F1-Score	Balance between Precision and Recall	2 × (P × R) / (P + R)

Confusion Matrix Explanation

```
              ┌──────────────────────────────┐
              │       PREDICTED              │
              │    YES (Car)   NO (Not Car)  │
┌─────────────┼──────────────────────────────┤
│     YES (Car) │  True Positive  │  False Negative  │
│   ACTUAL      │  (Correct)      │  (Missed)        │
│     NO (Car)  │  False Positive │  True Negative   │
│               │  (False Alarm)  │  (Correct)       │
```

7.2 Results Table

Metric	Faster R-CNN	YOLO
mAP (Average Precision)	**61.4%**	12.3%
mAP@0.5 (Standard IoU)	**94.9%**	52.0%
mAP@0.75 (Strict IoU)	**72.5%**	1.7%
Precision	**95.0%**	52.0%
Recall	**94.0%**	50.0%
F1-Score	**0.945**	0.510
Inference Time	~45 ms	~15 ms
Speed (FPS)	~22 FPS	~66 FPS